

1. Column

Definition

A layout composable that places its children vertically, one below another.

Proper Description

Column is the most commonly used parent container in Compose.

It works like a vertical LinearLayout from XML.

Every child added inside a Column is placed top to bottom in the order written.

You control spacing using verticalArrangement and alignment using horizontalAlignment.

Used for:

- Forms
- Lists
- Screens stacked vertically

Syntax :

```
Column(
    modifier = Modifier

        .fillMaxSize() // Fills the maximum available space

        .padding(16.dp), // Adds padding around the column

    verticalArrangement = Arrangement.Center, // Centers children vertically

    horizontalAlignment = Alignment.CenterHorizontally // Centers children horizontally
) {

    Text("Username")

    Text("Password")

}
```

2. Row

Definition

A layout composable that places its children horizontally, side by side.

Proper Description

Row behaves like a horizontal `LinearLayout`.
Children are arranged from left to right.

Very commonly used for:

- Toolbars
- Buttons in a single line
- Icon + Text combinations

You control spacing using `horizontalArrangement` and vertical alignment using `verticalAlignment`.

Syntax :

```
Row(  
    modifier = Modifier.fillMaxWidth(),  
    horizontalArrangement = Arrangement.SpaceBetween,  
    verticalAlignment = Alignment.CenterVertically  
) {  
    Text("Cancel")  
    Text("OK")  
}
```

3. Box

Definition

A layout composable that stacks children on top of each other.

Proper Description

Instead of arranging children in rows or columns, it places them in the same space, layered one above another.

Very useful for:

- Overlays
- Background + content
- Centering items

Syntax :

```
Box(  
    modifier = Modifier.fillMaxWidth(),  
    alignment = Alignment.Center,   
    contentPadding = 16.dp,  
    content = {  
        Text("Hello")  
    }  
)
```

```
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Text("Loading...")
    }
}
```

4. Parent Containers (Column, Row, Box)

Definition

Core layout building blocks used to structure the UI.

Proper Description

Jetpack Compose does not have complex layouts like ConstraintLayout by default. Instead, complex UIs are built by nesting Column, Row, and Box.

This makes layouts:

- Easier to read
- Easier to debug
- Easier to maintain

Syntax :

```
Column {
    Row {
        Icon(Icons.Default.Home, null)
        Text("Home")
    }
    Box {
        Text("Content Area")
    }
}
```

5. Floating Action Button (FAB)

Definition

A circular button representing the primary action of a screen.

Proper Description

FAB is meant for one main action only (e.g., Add, Create, Compose).
It floats above content and visually stands out.

Do NOT use multiple FABs on one screen.

Syntax :

```
FloatingActionButton(onClick = {}) {  
    Icon(Icons.Default.Add, contentDescription = "Add")  
}
```

6. Surface

Definition

A base container that defines background color, elevation, and shape.

Proper Description

Surface is a foundation composable in Material Design.
Many components (Card, Button) are built on top of it.

Use Surface when you want:

- Themed background
- Elevation
- Proper Material behavior

Syntax :

```
Surface(  
    color = MaterialTheme.colorScheme.primary,  
    modifier = Modifier.padding(16.dp)  
) {  
    Text("Surface Content", color = Color.White)  
}
```

7. Card

Definition

A Material container with elevation and rounded corners.

Proper Description

Card is used to group related information.
Cards create visual separation and hierarchy.

Typical uses:

- Product items
- Profile cards
- List items

Syntax :

```
Card(  
    elevation = CardDefaults.cardElevation(6.dp),  
    modifier = Modifier.padding(16.dp)  
) {  
    Text("Card Content", modifier = Modifier.padding(16.dp))  
}
```

8. Button

Definition

A clickable Material component used to trigger actions.

Proper Description

Buttons represent user actions.
Compose buttons are highly customizable and automatically follow Material theming.

Use buttons for:

- Submit
- Confirm
- Navigation actions

Syntax :

```
Button(onClick = {  
    println("Button was clicked!")  
}) {  
    Text("Click me")  
}
```

```
}  
}
```

9. Icon

Definition

Displays a vector or drawable icon.

Proper Description

Icons are visual indicators that improve usability.
They are commonly used inside buttons, toolbars, and lists.

Icons in Compose are vector-based and scalable.

Syntax :

```
Icon(  
    imageVector = Icons.Default.Star,  
    contentDescription = "Star Icon"  
)
```

Examples:

Action Icons

- Add (+) - Simple plus sign in a circle or square
- Delete (🗑️) - Trash can/bin icon
- Edit (✎) - Pencil or pencil tip
- Save (💾) - Floppy disk symbol
- Search (🔍) - Magnifying glass
- Share (↗) - Arrow pointing up-right or three connected dots

Alert Icons

- Error (❌) - Circle with "X" inside or exclamation in triangle
- Warning (⚠️) - Exclamation point in triangle
- Info (ℹ️) - Lowercase "i" in circle

Audio Icons

- Play (▶) - Right-pointing triangle

- Pause () - Two vertical bars
- Stop () - Square
- Volume () - Speaker with sound waves

Communication Icons

- Call () - Old telephone handset
- Email () - Envelope
- Message () - Speech bubble
- Chat () - Multiple speech bubbles or conversation bubbles

Content Icons

- Copy () - Two overlapping squares/pages
- Cut () - Scissors
- Paste () - Clipboard with paper
- Create () - Blank page with plus or folded corner

Device Icons

- Battery () - Battery with charge level
- Bluetooth (Bluetooth logo) - Runic "B" symbol
- WiFi () - Concentric arcs/signal bars

Editor Icons

- Attach () - Paperclip
- Insert (+) or () - Plus in circle or paperclip
- Format () - Underlined "A" or text formatting symbols

File Icons

- Folder () - Simple folder graphic
- Upload () - Upward arrow with line
- Download () - Downward arrow with line

Hardware Icons

- Keyboard () - Keyboard layout
- Mouse () - Computer mouse

- Phone (📱) - Smartphone outline

Image Icons

- Camera (📷) - Classic camera outline
- Photo (🖼️) - Mountain in rectangle (landscape photo)
- Filter (🖌️) - Funnel or brush/tilted square

Maps Icons

- Place (📍) - Map pin/drop pin
- Navigation (🧭) - Compass or directional arrow
- Location (📍) - Location dot/pin

Navigation Icons

- Menu (☰) - Hamburger menu (three horizontal lines)
- Home (🏠) - House outline
- Back (⬅️) - Left-pointing arrow or chevron
- Close (✕) - "X" symbol

Notification Icons

- Event (📅) - Calendar with day highlighted
- Alarm (🕒) - Bell or clock with bell
- Reminder (🔔) - Bell

Places Icons

- Business (🏢) - Building/office building
- Home (🏠) - House
- Work (🧑💼) - Briefcase or office building

Social Icons

- Person (👤) - Silhouette of head and shoulders
- Group (👥) - Multiple silhouettes
- Share (➦) - Arrow or connected nodes

Toggle Icons

- Check (✓) - Checkmark

- Radio (🔴) - Radio button (circle, often with dot inside when selected)
- Star (★) - Five-pointed star (filled or outlined)

10. State

Definition

A value that represents UI data and triggers recomposition.

Proper Description

State is the heart of Compose.

When a state value changes, Compose automatically rebuilds only the affected UI.

This eliminates:

- findViewById
- manual UI updates

Syntax :

`@Composable`

```
fun SimpleExample() {
    var typedText by remember { mutableStateOf("Type something...") }

    TextField(
        value = typedText,
        onValueChange = { newText -> typedText = newText }
    )

    Text("You typed: $typedText")
}
```

11. TextField

Definition

A Material-styled text input field.

Proper Description

TextField is used for user input such as names, emails, and passwords. It comes with built-in Material styling and behavior.

Syntax :

```
var text by remember { mutableStateOf("") }
```

```
TextField(  
    value = text,  
    onChange = { text = it },  
    label = { Text("Name") }  
)
```

12. BasicTextField

Definition

A minimal text input without Material styling.

Proper Description

BasicTextField gives full control to developers. You must manually handle styling, borders, and placeholders.

Used in:

- Custom UI designs
- Games
- Highly customized forms

Syntax :

```
BasicTextField(  
    value = text,  
    onChange = { text = it }  
)
```

13. OutlinedTextField

Definition

A text field with an outlined border.

Proper Description

OutlinedTextField is preferred when:

- Multiple fields exist on a screen
- Clear visual separation is needed

Very common in login and registration screens.

Syntax :

```
OutlinedTextField(  
    value = text,  
    onValueChange = { text = it },  
    label = { Text("Email") }  
)
```

14. Dropdown Menu

Definition

A popup menu displaying multiple options.

Proper Description

Dropdown menus are used to present secondary actions.
They must always be anchored to a parent component.

Syntax :

```
var expanded by remember { mutableStateOf(false) }
```

```
Box {  
    IconButton(onClick = { expanded = !expanded }) {  
        Icon(Icons.Default.ArrowDropDown, contentDescription = "Menu")  
    }  
}
```

```
DropdownMenu(  
    expanded = expanded,  
    onDismissRequest = { expanded = false }  
) {
```

```
DropdownMenuItem(  
    text = { Text("Option 1") },  
    onClick = { expanded = false }  
)  
  
DropdownMenuItem(  
    text = { Text("Option 2") },  
    onClick = { expanded = false }  
)  
}  
}
```

15. Spacer

Definition

An invisible component that adds empty space.

Proper Description

Spacer improves layout readability.
It avoids using empty Texts or padding hacks.

Syntax :

```
Spacer(modifier = Modifier.height(16.dp))
```

16. Padding

Definition

Adds spacing around or inside components.

Proper Description

Padding improves UI clarity and touch comfort.
Almost every Compose UI uses padding.

Syntax :

```
Modifier.padding(16.dp)
```

MODIFIERS (BEHAVIOR + LOOK)

1.padding()

Adds space around a composable.

```
Modifier.padding(8.dp)
```

2.fillMaxSize()

Makes a composable occupy all available space.

```
Modifier.fillMaxSize()
```

3.size()

Sets exact width and height.

```
Modifier.size(100.dp)
```

4. background()

Applies background color or shape.

```
Modifier.background(Color.Gray)
```

5.clickable()

Makes a composable respond to clicks.

```
Modifier.clickable { }
```

6.weight()

Distributes available space in Row/Column.

```
Row {  
    Text("A", Modifier.weight(1f))  
    Text("B", Modifier.weight(2f))  
}
```

PREVIEW, CONTEXT & TOAST

1. Preview

Definition

Shows UI without running the app.

Proper Description

Preview speeds up UI development and learning.
Multiple previews can be created for different devices.

```
@Preview(showBackground = true)
```

```
@Composable
```

```
fun PreviewUI() {
```

```
    Text("Preview")
```

```
}
```

2. Context

Definition

Provides access to Android system services.

Proper Description

Context is needed for:

- Toast
- Resources
- Navigation
- System services

```
val context = LocalContext.current
```

3. Toast

Definition

A short popup message.

Proper Description

Toasts provide quick feedback without blocking the UI.

```
Toast.makeText(  
    context,  
    "Saved Successfully",  
    Toast.LENGTH_SHORT  
)
```

```
.show()
```